

VERSION CONTROL · FROM ZERO TO PROFESSIONAL

PREMIUM
EDITION

GIT & GITHUB HANDBOOK

Learn Version Control Like a Professional

A complete, beginner-to-advanced guide covering Git internals, branching, rebasing, GitHub collaboration, real interview questions, and the habits that separate hobbyists from professional engineers.

Created by Hammy Explains

YouTube: [Hammy Explains](#) • Instagram: [@hammy.explains](#)

TABLE OF CONTENTS

What's Inside

Ch.	Title
1	Introduction — What is Version Control?
2	The Start — A Short History of Git
3	What is Git?
4	What is GitHub?
5	Git and GitHub Together
6	Installation and Setup
7	The Git Workflow
8	Basic Git Commands
9	Branches in Git
10	Git Reflog — Your Safety Net
11	Merge Conflicts
12	Tags in Git
13	Git Stash
14	Git Rebase
15	GitHub Setup & Authentication
16	Git Using VS Code
17	The Final OG Concepts
18	The Patriot — Professional Developer Mindset
19	Git Cheat Sheet
20	50 Interview Questions
21	Practical Projects
22	Final Roadmap

Plus: tip boxes, warning boxes, pro tips, common mistakes, real-world scenarios, chapter summaries, and knowledge checks throughout.

CHAPTER 01

Introduction

What is Version Control, and why does every developer need it?

The Problem Before Version Control

Imagine you're writing a college assignment in Word. You save it. You make changes. You're not sure if the changes are good, so you save a copy just in case. A week later, your folder looks like this:

```
project_final.py
project_final_final.py
project_final_latest.py
project_final_latest_ACTUAL.py
project_final_latest_ACTUAL_v2.py
```

COMMON MISTAKE

This isn't a joke — nearly every beginner does this before discovering Git. The problem isn't laziness; it's that there was no better system available. Naming files by hand doesn't scale, and it completely breaks down the moment more than one person touches the project.

Now imagine this happening on a real software team: five engineers, one codebase, all editing the same files at the same time. Whose "final" version wins? What if two people fixed the same bug in different ways? What if someone accidentally deletes a working feature and nobody notices for two weeks?

Why Developers Needed Something Better

Teams needed a system that could:

- Track every change made to every file, automatically
- Let multiple people work on the same project without overwriting each other
- Allow you to go back in time to any previous version
- Show exactly *who* changed *what*, and *when*, and *why*
- Let people experiment safely without breaking the main project

This is **Version Control**: a system that records changes to a set of files over time, so you can recall specific versions later.

REAL-WORLD ANALOGY

Think of Version Control like the "track changes" and "version history" feature in Google Docs — but supercharged for code, working offline, able to merge everyone's edits intelligently, and built to handle thousands of files across a team of any size.

Enter Git

Git is the tool that solved this problem, and it solved it so well that it became the industry standard. Whether you join a startup, a product-based company, or an open-source project, Git is the language every team speaks. This handbook will take you from "I've never opened a terminal" to "I can confidently manage version control on a real engineering team."

PRO TIP

You don't need to memorize every Git command on day one. Professionals use roughly 15–20 commands for 95% of their daily work. This handbook focuses on exactly those, then builds up to the advanced 5% that make you look like a senior engineer.

INTERVIEW QUESTION**Q: What problem does version control solve?**

A: It solves the problem of tracking, coordinating, and recovering changes to a project over time — especially when multiple people are editing the same files. Without it, teams rely on manual file naming and copies, which is error-prone and doesn't scale.

Chapter Summary

- Manually naming files (final, final_v2) does not scale and breaks on real teams
- Version control automatically tracks every change to every file
- Git is the industry-standard version control system

KNOWLEDGE CHECK

1. Why does manual file naming fail on team projects?
2. Name three things a good version control system should do.
3. What analogy from this chapter describes version control?

CHAPTER 02

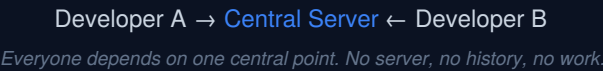
The Start

A short history of Git — and why it was built the way it was

Before Git: The Old World

Long before Git existed, teams used centralized version control systems like CVS and Subversion (SVN). These systems stored the entire project history on one central server. Every developer had to be connected to that server to commit changes, view history, or create branches. If the server went down, work stopped.

Centralized Model (Old Way)



Linus Torvalds and the Birth of Git

Linus Torvalds, the creator of the Linux kernel, needed a way for thousands of contributors around the world to collaborate on the Linux source code without a single central bottleneck. In 2005, after a licensing dispute with the proprietary tool the Linux project had been using, Torvalds spent about ten days writing the first version of Git himself.

REAL-WORLD ANALOGY

Centralized version control is like a single shared Google Doc that everyone must be online to edit. Git is like everyone having their own full offline copy of the entire document's history — and a reliable way to merge everyone's edits back together later.

Why Git Was Designed Differently

- **Distributed:** every developer has a full copy of the project history
- **Fast:** almost every operation happens locally, instantly
- **Non-linear workflows:** built for thousands of parallel branches
- **Data integrity:** every change is checksummed (SHA-1/SHA-256), so corruption or tampering is detectable

Evolution of Version Control

Era	Tool	Model
1980s–90s	RCS, CVS	Centralized, file-by-file locking
2000s	Subversion (SVN)	Centralized, whole-project versioning
2005–present	Git, Mercurial	Distributed version control
2008–present	GitHub, GitLab, Bitbucket	Git + cloud collaboration

REAL-WORLD SCENARIO

Today, Git isn't just for open source. Google, Microsoft, Amazon, Netflix, and virtually every modern tech company use Git (often via GitHub, GitLab, or an internal fork) as the backbone of how code moves from a developer's laptop into production.

INTERVIEW QUESTION

Q: Why is Git called a "distributed" version control system?

A: Because every developer clones a full copy of the repository, including its entire history, rather than depending on a single central server for every operation.

Chapter Summary

- Old centralized systems (CVS, SVN) depended on one central server
- Linus Torvalds created Git in 2005 to support Linux kernel development at scale
- Git is distributed, fast, and built for non-linear, parallel collaboration

CHAPTER 03

What is Git?

Local version control vs. distributed version control

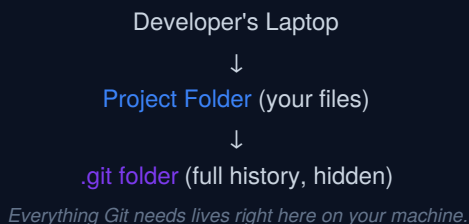
Definition

Git is a free, open-source, **distributed version control system** designed to track changes in source code (or any set of files) during software development. It runs entirely on your machine — no internet connection required for most operations.

Local Version Control

Git lives inside a hidden folder called `.git` in your project directory. This folder contains the entire history of your project: every commit, every branch, every version of every file that was ever saved. This is what makes Git "local" — your repository is fully functional on your own laptop, with no server needed.

Local Git Repository



Distributed Version Control

Because every clone of a Git repository contains the complete history, Git is "distributed" — there is no single point of failure. If a central GitHub server were to disappear tomorrow, any developer's local clone could be pushed to a new server and the entire project history would be restored.

Developer ↔ **Git Repository** (full history, local)

Every developer has the complete project — not just the latest snapshot.

Practical Example

```
cd my-project
git init           # turns this folder into a Git repository
ls -la            # notice the new .git folder
```

REAL-WORLD ANALOGY

A centralized system is like one master recipe book locked in a restaurant's kitchen. Git is like giving every chef their own complete copy of the recipe book, including every past edition, so they can cook (and even revise recipes) even if the restaurant loses power.

Benefits of Git

- Works fully offline — commit, branch, and view history without internet
- Extremely fast, since almost everything is a local disk operation
- Cheap, instant branching — encourages experimentation
- Full history is always available, even if the remote server is down
- Strong data integrity through cryptographic hashing of every commit

INTERVIEW QUESTION

Q: What's the difference between local and distributed version control?

A: Local version control tracks history on a single machine. Distributed version control (what Git does) gives every collaborator a full copy of the repository and its history, removing the need for constant server access.

Chapter Summary

- Git stores your entire project history in a hidden `.git` folder
- Because every clone has the full history, Git has no single point of failure
- Benefits: offline-first, fast, cheap branching, strong data integrity

CHAPTER 04

What is GitHub?

Git vs. GitHub — local vs. cloud, and why collaboration needs both

Git vs. GitHub: The Confusion Every Beginner Has

This is one of the most common points of confusion for beginners:**Git and GitHub are not the same thing.**

- **Git** is the version control tool — software that runs on your computer and tracks changes.
- **GitHub** is a website/cloud platform that hosts Git repositories online, adding collaboration features on top of Git.

REAL-WORLD ANALOGY

Git is like Microsoft Word — the tool you use to write and track changes in a document. GitHub is like Google Drive or Dropbox — a place to store that document online, share it with others, and let a team collaborate on it.

Local vs. Cloud



You could use Git without ever touching GitHub — plenty of solo developers do. But the moment you want to collaborate, back up your code remotely, or contribute to open source, you need a hosting platform like GitHub (or alternatives like GitLab and Bitbucket, which work almost identically).

What GitHub Adds on Top of Git

Feature	Provided By
Tracking file changes, commits, branches	Git
Cloud storage of repositories	GitHub
Pull Requests & code review	GitHub
Issue tracking	GitHub
Team permissions & collaborators	GitHub
CI/CD automation (GitHub Actions)	GitHub
Project boards, discussions, wikis	GitHub

Git vs. GitHub: Quick Comparison

Git	GitHub
A tool (software)	A platform (website/service)
Installed locally on your machine	Hosted in the cloud
Works fully offline	Requires internet access
Created by Linus Torvalds (2005)	Founded in 2008, now owned by Microsoft
No built-in UI for code review	Pull requests, review comments, discussions

WARNING

Deleting a GitHub repository does *not* delete Git itself, and deleting your local `.git` folder does not affect what's stored on GitHub. They are separate systems — understanding this distinction will save you from a lot of panic later.

INTERVIEW QUESTION

Q: Can you use Git without GitHub?

A: Yes. Git is fully functional on its own for local version control. GitHub is optional and is used for cloud hosting, backup, and team collaboration on top of Git.

Chapter Summary

- Git = the version control tool. GitHub = the cloud platform that hosts Git repos
- GitHub adds collaboration features: pull requests, issues, code review, CI/CD
- You can use Git without GitHub, but not GitHub without Git

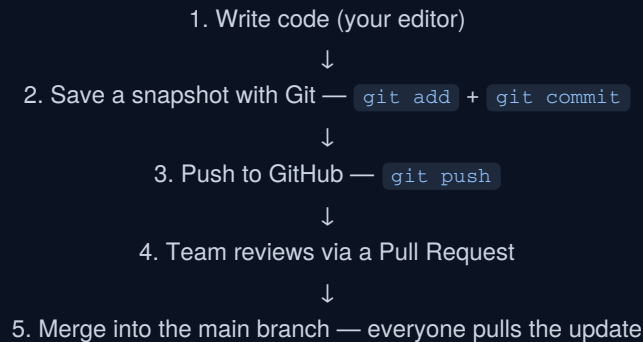
CHAPTER 05

Git and GitHub Together

How they work as one complete system in a real project

The Complete Relationship

In practice, Git and GitHub work together as a single pipeline that takes code from your editor to your teammates' screens. Here is the full journey of a single change:



This loop repeats hundreds of times a day on a real engineering team.

Real-World Project Example

Say you're building a small e-commerce site with two teammates. You're assigned the "checkout page." Here's what your day looks like:

```
git checkout -b feature-checkout-page # create your own branch
# ... write code ...
git add .
git commit -m "feat: add checkout page layout"
git push origin feature-checkout-page # send it to GitHub
```

On GitHub, you open a **Pull Request** asking to merge `feature-checkout-page` into `main`. A teammate reviews your code, leaves comments, you push a few fixes, and once approved, it's merged. Everyone else runs `git pull` to get your changes.

PRO TIP

Notice that steps 1–2 (writing code, committing) never require the internet. Only step 3 (push) and step 5 (pull) need GitHub. This is exactly why Git is fast — almost everything happens locally, and syncing with GitHub is a deliberate, occasional action.

REAL-WORLD SCENARIO

This exact loop — branch, commit, push, PR, review, merge, pull — is used at nearly every tech company, from two-person startups to companies with thousands of engineers. Learning it well is one of the highest-leverage skills you can build early in your career.

INTERVIEW QUESTION

Q: Walk me through what happens when you push code to GitHub.

A: Your local commits (already saved in your Git history) are uploaded to the corresponding branch on the GitHub-hosted copy of the repository, making them visible and accessible to your team.

Chapter Summary

- Write → commit (Git, local) → push (to GitHub) → Pull Request → merge → pull
- Writing and committing code never require internet; pushing and pulling do
- This cycle is the daily rhythm of real professional software teams

CHAPTER 06

Installation and Setup

Getting Git running on Windows, Mac, and Linux

Installing Git

Windows

1. Download the installer from git-scm.com
2. Run the installer — the default options work fine for beginners
3. This installs **Git Bash**, a terminal that behaves like Linux/Mac

Mac

```
# Option 1: via Homebrew (recommended)
brew install git

# Option 2: Xcode Command Line Tools (also installs Git)
xcode-select --install
```

Linux

```
# Debian / Ubuntu
sudo apt update && sudo apt install git

# Fedora
sudo dnf install git

# Arch
sudo pacman -S git
```

Verify Installation

```
git --version
git version 2.43.0
```

If you see a version number, Git is installed correctly. If you get a "command not found" error, restart your terminal — sometimes the installer needs a fresh terminal session to update your system's PATH.

Configure Git (One-Time Setup)

Before your first commit, Git needs to know who you are. This information gets attached to every commit you make, forever.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Line-by-Line Explanation

- `git config` — the command used to set Git's settings
- `--global` — applies this setting to every repository on your machine (not just the current one)
- `user.name` / `user.email` — the identity attached to your commits

You can verify your settings at any time:

```
git config --global --list
```

WARNING

Use the same email here that you use for your GitHub account. If they don't match, GitHub won't correctly link your commits to your profile, and your green contribution squares won't show up.

TIP

You can override the global config for a specific project by running the same commands *without* `--global` inside that project's folder — useful if you use a different email for work vs. personal projects.

INTERVIEW QUESTION

Q: What does `git config --global` do?

A: It sets a configuration value (like your name or email) that applies to every Git repository on your machine, rather than just the current project.

Chapter Summary

- Install Git via the official installer (Windows) or a package manager (Mac/Linux)
- Verify with `git --version`
- Set your identity once with `git config --global user.name/user.email`

CHAPTER 07

The Git Workflow

Understanding the four stages every change passes through

The Four Stages

Every file in a Git project moves through a predictable pipeline. Understanding this pipeline is the single most important mental model in this entire handbook — almost every Git command simply moves a file from one stage to the next.

Working Directory — where you edit files

↓ `git add`

Staging Area — changes marked "ready to save"

↓ `git commit`

Local Repository — permanently saved snapshot

↓ `git push`

Remote Repository — shared copy on GitHub

Four stages: Working Directory → Staging Area → Local Repo → Remote Repo

1. Working Directory

This is simply your project folder — the actual files you see and edit in your code editor. Git watches this folder for changes but doesn't save anything automatically.

2. Staging Area (the "index")

Think of staging as a shopping cart. You can add items to it, remove them, and review what's inside before "checking out." `git add` moves changes from your working directory into this cart.

```
git add app.py      # stage a single file
git add .           # stage everything that changed
```

3. Local Repository

`git commit` takes everything in the staging area and permanently saves it as a snapshot in your project's history, along with a message describing what changed.

```
git commit -m "feat: add user login form"
```

4. Remote Repository

`git push` uploads your local commits to GitHub (or whatever remote you're using), making them visible to your team.

```
git push origin main
```

REAL-WORLD ANALOGY

Packing for a trip: your **working directory** is your whole room. Your **staging area** is the open suitcase on the bed — you

deliberately choose what goes in. Your **local repository** is the suitcase, zipped and ready. Your **remote repository** is the suitcase after it's checked in at the airport — now shared with the "system" (your team).

COMMON MISTAKE

Beginners often run `git commit` and expect their code to already be on GitHub. Remember: commit only saves locally. You must `git push` separately to share it.

INTERVIEW QUESTION

Q: What's the difference between `git add` and `git commit` ?

A: `git add` stages changes — marking them as ready to be saved. `git commit` actually saves those staged changes as a permanent snapshot in the repository's history.

Chapter Summary

- Every change flows: Working Directory → Staging → Local Repo → Remote Repo
- `git add` stages, `git commit` saves locally, `git push` shares
- This four-stage model explains almost every Git command in this book

CHAPTER 08

Basic Git Commands

The everyday toolkit — syntax, examples, and use cases

git init

Purpose: turns any folder into a Git repository.

```
git init
Initialized empty Git repository in /project/.git/
```

Use case: run once, at the very start of a new project.

git status

Purpose: shows what has changed — staged, unstaged, and untracked files.

```
git status
On branch main
Changes not staged for commit:
  modified:   app.py
Untracked files:
  notes.txt
```

PRO TIP

Run `git status` constantly — before and after almost every command. It's the single best habit for avoiding Git mistakes.

git add

Purpose: stages changes for the next commit.

```
git add app.py      # stage one file
git add src/        # stage a folder
git add .           # stage everything
```

git commit

Purpose: saves staged changes as a permanent snapshot.

```
git commit -m "fix: correct total price calculation"
```

Use case: commit often, in small logical chunks — not once at the end of the day.

git log

Purpose: shows the commit history.

```
git log --oneline --graph
* a1b2c3d (HEAD -> main) fix: correct total price
* 9f8e7d6 feat: add checkout page
```

git diff

Purpose: shows exact line-by-line changes that haven't been staged yet.

```
git diff
-price = qty * cost
+price = qty * cost * 1.18    # added tax
```

git rm

Purpose: removes a file from both the working directory and Git tracking.

```
git rm old_config.json
```

git mv

Purpose: renames or moves a file while preserving its Git history.

```
git mv app.py main.py
```

git restore

Purpose: discards unwanted changes, or unstages a file.

```
git restore app.py          # discard unstaged changes
git restore --staged app.py  # unstage (keep the edits)
```

WARNING

`git restore` (without `--staged`) permanently discards your uncommitted edits. There is no undo for this — always check `git status` and `git diff` first.

Quick Reference

Command	What it does
<code>git init</code>	Create a new repository

<code>git status</code>	Show current state of files
<code>git add</code>	Stage changes
<code>git commit</code>	Save staged changes
<code>git log</code>	View commit history
<code>git diff</code>	View exact changes
<code>git rm</code>	Remove a tracked file
<code>git mv</code>	Rename/move a tracked file
<code>git restore</code>	Discard or unstage changes

INTERVIEW QUESTION

Q: How do you undo changes to a file you haven't committed yet?

A: `git restore <file>` discards unstaged changes and reverts the file to its last committed state.

Chapter Summary

- These nine commands cover roughly 80% of daily Git usage
- `git status` and `git diff` are your safety checks — use them constantly
- `git restore` discards changes permanently — use with care

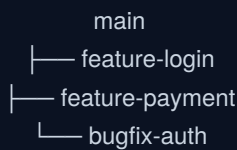
CHAPTER 09

Branches in Git

Working in parallel without breaking anything

Why Branches Exist

A branch is an independent line of development. Without branches, everyone would edit the same "main" version of the code at once — any half-finished feature would break the project for everyone. Branches let you build something new (or fix a bug) in isolation, without touching the stable, working code.



Each branch is an isolated line of work that starts from main.

Common Branch Types

- **main** — the stable, deployable version of the project
- **feature branch** — new functionality (e.g. `feature-login`)
- **bugfix branch** — a targeted fix (e.g. `bugfix-auth`)

Core Branch Commands

```
git branch                # list all local branches
git branch feature-login  # create a new branch
git checkout feature-login # switch to it (older command)
git switch feature-login  # switch to it (modern command)
git checkout -b feature-login # create AND switch in one step
git switch -c feature-login  # modern equivalent
```

TIP

`git switch` was introduced to replace the more confusing `git checkout`, which was overloaded to do too many different things (switch branches, restore files, and more). Modern teams prefer `switch` and `restore`.

Real-World Example

```
git switch -c feature-checkout-page
# ... write your code ...
git add .
git commit -m "feat: build checkout page"
git switch main
git merge feature-checkout-page
```

Deleting a Branch

```
git branch -d feature-login      # safe delete (only if merged)
git branch -D feature-login      # force delete (even if unmerged)
```

COMMON MISTAKE

New developers often make all their changes directly on `main`. On any real team, this is considered risky — always create a feature branch, even for small changes. It makes review, rollback, and collaboration dramatically easier.

PRO TIP

Name branches descriptively and consistently, e.g. `feature/checkout-page`, `bugfix/login-crash`, `hotfix/payment-error`. This convention is covered in depth in Chapter 18.

INTERVIEW QUESTION

Q: What's the difference between `git branch` and `git checkout -b` ?

A: `git branch <name>` only creates a new branch. `git checkout -b <name>` creates the branch *and* switches to it immediately.

Chapter Summary

- Branches let you build in isolation without breaking the stable main branch
- `git switch -c` is the modern way to create and move to a new branch
- Real teams almost never commit directly to main

CHAPTER 10

Git Reflog

Your safety net — recovering from almost any Git disaster

What is Reflog?

Every time `HEAD` moves — a commit, a checkout, a merge, even a reset — Git silently records it in the **reference log** (reflog). This log is local, private to your machine, and kept for about 90 days by default. It's essentially Git's own undo history for itself.

Why It's Powerful

Reflog can recover commits that seem to be permanently lost — even after a hard reset, a deleted branch, or an "oops" moment that looks unrecoverable to a beginner.

```
git reflog
a1b2c3d HEAD@{0}: commit: fix login bug
9f8e7d6 HEAD@{1}: checkout: moving from main to feature-login
4d5e6f7 HEAD@{2}: reset: moving to HEAD~1
```

Recovering "Deleted" Commits

Say you accidentally ran `git reset --hard HEAD~1` and lost a commit. Here's how to get it back:

```
git reflog          # find the commit hash before the mistake
git reset --hard a1b2c3d # restore HEAD to that exact commit
```

Commit made → Accidental hard reset → "Lost" → `git reflog` → Found → `git reset --hard <hash>` → Recovered

PRO TIP

Reflog is why experienced developers rarely panic over Git mistakes. Almost nothing is truly gone as long as it was committed at some point — reflog keeps a trail even after resets, deleted branches, and rebases.

REAL-WORLD SCENARIO

A teammate force-pushes over a branch, wiping out three days of a colleague's commits from the remote. As long as the colleague's *local* reflog still has the commits, they can recover them and push again — a genuinely common "saved my job" story in engineering teams.

WARNING

Reflog is local only — it is not pushed to GitHub and is not shared with your team. If you never committed the work in the first place (or you're on a fresh clone), reflog cannot help you.

INTERVIEW QUESTION

Q: How would you recover a commit after an accidental `git reset --hard` ?

A: Run `git reflog` to find the commit hash from before the reset, then run `git reset --hard <hash>` to restore it.

Chapter Summary

- Reflog silently logs every movement of HEAD on your local machine
- It can recover commits lost to resets, deleted branches, and more
- Reflog is local-only and typically expires after ~90 days

CHAPTER 11

Merge Conflicts

What they are, why they happen, and how to resolve them calmly

What is a Merge Conflict?

A merge conflict happens when Git cannot automatically combine changes from two branches — usually because two people edited the *same line* of the *same file* in different ways, and Git doesn't know which version you want.

Why Does It Happen?

Developer A

```
discount = 0.10
```

Developer B

```
discount = 0.15
```

Both branches changed the same line differently. When Developer B tries to merge into main after Developer A's change is already there, Git stops and asks a human to decide.

What It Looks Like

```
<<<<<<< HEAD
discount = 0.10
=====
discount = 0.15
>>>>>>> feature-discount
```

- `<<<<<<< HEAD` — start of your current branch's version
- `=====` — the dividing line
- `>>>>>>> feature-discount` — end of the incoming branch's version

Resolving the Conflict

1. Open the file — Git marks every conflicted section like the example above
2. Decide what the correct final code should be (keep one side, the other, or a mix of both)
3. Delete the `<<<<<<<`, `=====`, and `>>>>>>>` markers
4. Stage and commit the resolved file

```
git add pricing.py
git commit -m "merge: resolve discount conflict, use 0.15"
```

REAL-WORLD ANALOGY

Think of two editors making different handwritten corrections on the same sentence of a shared manuscript. Someone has to sit down, look at both corrections, and decide the final wording. That's exactly what conflict resolution is — a human decision Git can't make for you.

Best Practices

- Pull from main frequently so your branch doesn't drift too far before merging
- Keep branches short-lived and focused on one feature
- Communicate with teammates when you know you're both touching the same file
- Use your editor's merge tool (VS Code highlights conflicts with clickable buttons)

COMMON MISTAKE

Panicking and running `git merge --abort` repeatedly without ever resolving the underlying conflict just delays the problem. Take a breath, read the conflict markers carefully — they're simpler than they look.

INTERVIEW QUESTION

Q: What causes a merge conflict, and how do you resolve one?

A: A conflict occurs when Git can't automatically reconcile changes to the same lines in two branches. You resolve it by manually editing the conflicted file to keep the correct code, removing the conflict markers, then staging and committing the result.

Chapter Summary

- Conflicts happen when the same lines are changed differently on two branches
- Git marks conflicts with `<<<<<<`, `=====`, `>>>>>>`
- Resolve manually, then `git add` and `git commit`

CHAPTER 12

Tags in Git

Marking release points in your project's history

Purpose of Tags

A tag is a permanent, named pointer to a specific commit — typically used to mark release versions. Unlike branches, tags don't move; once created, a tag always points to the exact same commit.

Release Versions

Most projects follow **Semantic Versioning**: `MAJOR.MINOR.PATCH`

```
v1.0.0    # first stable release
v1.1.0    # new feature added, backward-compatible
v2.0.0    # breaking change
```

Creating Tags

```
git tag v1.0.0                # lightweight tag
git tag -a v1.0.0 -m "First stable release"  # annotated tag (recommended)
```

TIP

Prefer annotated tags (`-a`) for releases — they store the tagger's name, date, and message, which is important documentation for a real release history.

Pushing Tags to GitHub

Tags are not pushed automatically with `git push` — they need an explicit push:

```
git push origin v1.0.0        # push a single tag
git push origin --tags        # push all tags
```

Release Workflow

Feature branches merged into main

↓

main is stable and tested

↓

```
git tag -a v1.2.0 -m "release notes"
```

↓

```
git push origin v1.2.0
```



GitHub Release created from the tag

PRO TIP

On GitHub, you can turn any tag into a formal "Release" with changelog notes and downloadable assets — this is how most open-source projects publish new versions.

INTERVIEW QUESTION

Q: What's the difference between a branch and a tag?

A: A branch is a movable pointer that advances as you commit. A tag is a fixed pointer to one specific commit — it never moves, making it ideal for marking releases.

Chapter Summary

- Tags mark permanent, named points in history — typically releases
- Use annotated tags (`git tag -a`) for real projects
- Tags must be pushed explicitly with `git push origin --tags`

CHAPTER 13

Git Stash

Temporarily saving unfinished work

Why Stash Exists

Imagine you're halfway through a messy, uncommitted change when your teammate asks you to urgently switch branches to fix a production bug. You're not ready to commit your half-finished work — but Git won't let you switch branches cleanly with conflicting uncommitted changes in the way. `git stash` solves this by temporarily "shelving" your changes.

REAL-WORLD ANALOGY

Stashing is like sweeping your messy desk into a drawer before an unexpected guest arrives — nothing is lost, it's just set aside so you can deal with something urgent, and you can pull it back out exactly as it was later.

Core Commands

```
git stash          # save current changes, clean the working directory
git stash list     # view all stashed changes
git stash pop      # restore the most recent stash and remove it from the list
git stash apply    # restore the most recent stash but KEEP it in the list
git stash drop     # delete a stash without restoring it
```

Real-World Example

```
git stash save "WIP: checkout page styling"
git switch main
git switch -c hotfix-payment-crash
# ... fix the urgent bug, commit, push ...
git switch feature-checkout-page
git stash pop      # your unfinished work is back exactly as you left it
```

Stashing Specific Files

```
git stash push app.py styles.css # stash only these files
```

PRO TIP

Always give your stashes descriptive messages with `git stash save "message"`. A list of ten unnamed stashes is nearly impossible to make sense of later.

COMMON MISTAKE

Forgetting stashed changes exist. If something you swore you wrote seems to have vanished, always check `git stash list` before assuming it's lost.

INTERVIEW QUESTION

Q: What's the difference between `git stash pop` and `git stash apply` ?

A: Both restore stashed changes to your working directory. `pop` also removes the stash from the stash list afterward; `apply` keeps it there so you can reuse it elsewhere.

Chapter Summary

- Stash temporarily saves uncommitted changes so you can switch context cleanly
- `git stash pop` restores and removes; `apply` restores and keeps
- Name your stashes — a nameless stash list is confusing later

CHAPTER 14

Git Rebase

Merge vs. rebase — two ways to combine history

What is Rebase?

Rebasing takes the commits from your branch and replays them on top of another branch's latest commit, creating a clean, linear history — as if you had started your work *after* the latest changes on main, rather than before.

Merge vs. Rebase

Merge

```
main: A---B-----M
      \   /
feature: C---D
```

Preserves full history exactly as it happened, including a merge commit.

Rebase

```
main: A---B
feature:   \
           C'---D'
```

Rewrites your commits on top of B, producing a straight line of history.

Commands

```
git switch feature-login
git rebase main           # replay feature-login's commits on top of main
```

If conflicts occur mid-rebase, Git pauses so you can resolve them one commit at a time:

```
# resolve the conflict in the file, then:
git add .
git rebase --continue

# or, to cancel entirely:
git rebase --abort
```

Benefits

- Produces a clean, linear, easy-to-read project history
- Makes `git log` and `git bisect` much easier to follow
- Avoids clutter from dozens of small "merge branch main into feature" commits

Risks

- Rebasing rewrites commit history — the original commits are replaced with new ones
- Rebasing commits that others have already pulled can cause serious confusion for your team

WARNING — THE GOLDEN RULE OF REBASE

Never rebase commits that have already been pushed and shared with others. Rebasing rewrites commit hashes; if teammates have already based work on the old commits, you'll create a painful, confusing mess. Only rebase your own local, unshared work.

Best Practices

- Rebase your local feature branch before opening a Pull Request, to keep history clean
- Never rebase `main` or any shared branch
- When in doubt, merge — it's always safe

INTERVIEW QUESTION

Q: When would you use rebase instead of merge?

A: Use rebase to keep a clean, linear history on your own local, unpushed branch before opening a PR. Avoid rebasing shared/pushed branches, since it rewrites commit history that others may already depend on.

Chapter Summary

- Merge preserves history exactly; rebase rewrites it into a straight line
- Rebase is powerful but dangerous on shared branches — never rebase what others have pulled
- Golden rule: rebase locally, merge publicly

CHAPTER 15

GitHub Setup

Accounts, repositories, cloning, and authentication

Create an Account

Sign up at github.com with a professional username — this profile is often the first thing recruiters and interviewers check, so choose something you'd be comfortable sharing on a resume.

Create a Repository

1. Click "New Repository" on GitHub
2. Name it, add a description, choose Public or Private
3. Optionally initialize with a README, `.gitignore`, and license

Clone a Repository

```
git clone https://github.com/username/repo-name.git
```

This downloads the entire project — including full history — onto your machine, already connected to GitHub as its "origin" remote.

Connect an Existing Local Project to GitHub

```
git init
git remote add origin https://github.com/username/repo-name.git
git branch -M main
git push -u origin main
```

`-u` sets `origin main` as the default upstream, so future pushes only need `git push`.

Authentication Methods

Personal Access Tokens (PAT)

GitHub no longer accepts your account password for Git operations over HTTPS. Instead, you generate a token (Settings → Developer Settings → Personal Access Tokens) and use it in place of a password when prompted.

SSH Keys

```
ssh-keygen -t ed25519 -C "you@example.com" # generate a key pair
cat ~/.ssh/id_ed25519.pub                  # copy the public key
# paste it into GitHub → Settings → SSH and GPG Keys
```

```
git remote set-url origin git@github.com:username/repo-name.git
```

TIP

SSH keys are the more convenient long-term choice — once set up, you authenticate automatically without repeatedly entering a token.

Security Best Practices

- Never commit passwords, API keys, or tokens into your repository
- Use a `.gitignore` to exclude secrets and environment files
- Enable two-factor authentication (2FA) on your GitHub account
- Rotate tokens periodically and give them the minimum scopes they need

WARNING

If you accidentally commit a secret, changing it later isn't enough — the old value remains visible in Git history forever unless you rewrite history. Revoke and rotate the exposed credential immediately.

INTERVIEW QUESTION

Q: Why doesn't GitHub accept your account password for Git operations anymore?

A: For security, GitHub requires a Personal Access Token or SSH key instead of a raw account password, limiting the scope and revocability of credentials used for Git operations.

Chapter Summary

- `git clone` downloads a full repository, already connected to GitHub
- Authenticate with a Personal Access Token or, preferably, an SSH key
- Never commit secrets; use `.gitignore` and 2FA

CHAPTER 16

Git Using VS Code

A visual workflow for everything you've learned so far

The Source Control Panel

VS Code has Git support built in. Click the branch-shaped icon in the left sidebar (or press `Ctrl+Shift+G`) to open the Source Control panel, which shows every changed file, grouped by staged and unstaged.

[Source Control Icon]



Changes list — click "+" next to a file to stage it (same as `git add`)



Message box at top — type a commit message, press Ctrl+Enter (same as `git commit`)

Step-by-Step Guide

1. **Commit:** stage files with the "+" icon, type a message, click the checkmark (✓)
2. **Push:** click "..." menu → Push, or use the sync icon in the status bar
3. **Pull:** same "..." menu → Pull, to bring down teammates' changes
4. **Sync:** the circular arrows icon does a pull followed by a push in one click
5. **Branch Management:** click the branch name in the bottom-left status bar to switch, create, or delete branches visually

TIP

VS Code highlights uncommitted changes directly in the editor gutter — a green bar for added lines, a blue bar for modified lines, and a red triangle for deleted lines. This is often faster to scan than `git diff` in a terminal.

Resolving Conflicts Visually

When a merge conflict occurs, VS Code shows clickable buttons above each conflicted block: **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**, and **Compare Changes** — removing the need to manually edit the raw conflict markers from Chapter 11.

PRO TIP

Even when using the VS Code UI daily, keep practicing the terminal commands from earlier chapters. Interviews and remote servers (where there's no GUI) will expect you to know the underlying Git commands, not just the buttons.

COMMON MISTAKE

Relying on VS Code's UI without ever learning what each button actually does under the hood makes it very hard to debug when something looks wrong or a button is greyed out. Always know the terminal command each button maps to.

INTERVIEW QUESTION

Q: Does using a GUI like VS Code change how Git actually works?

A: No — GUIs are a visual layer over the same underlying Git commands (`add` , `commit` , `push` , etc.). Understanding the CLI is still essential, especially for servers and CI environments without a GUI.

Chapter Summary

- The Source Control panel maps directly to `add` , `commit` , `push` , `pull`
- Conflict resolution is visual, with one-click accept buttons
- Still learn the underlying commands — GUIs are a convenience layer, not a replacement

CHAPTER 17

The Final OG Concepts

Don't miss these — the concepts that unlock real open-source work

.gitignore

A file that tells Git which files/folders to never track — build artifacts, dependencies, secrets, OS files.

```
# .gitignore
node_modules/
.env
__pycache__/
*.log
.DS_Store
```

README.md

The front door of your repository — the first thing anyone sees. A good README includes: what the project does, how to install it, how to run it, and how to contribute.

Fork

A fork creates your own personal copy of someone else's repository on GitHub, letting you make changes without needing write access to the original project.

Clone

Downloads a copy of a repository (yours, a fork, or any public repo) onto your local machine so you can work on it.

Original Repo → **Fork** (your GitHub copy) → **Clone** (your laptop)

Pull Request (PR)

A formal request asking the maintainers of a repository to merge your changes into theirs. This is the core mechanism of GitHub collaboration and open source.

Code Review

The process of teammates or maintainers reading your PR's code, leaving comments, requesting changes, and eventually approving it before merge.

Issues

GitHub's built-in bug/feature tracker — used to report bugs, propose features, and discuss work before it's coded.

The Open Source Workflow

1. Find an open-source project → 2. Fork it → 3. Clone your fork → 4. Create a branch → 5. Make changes → 6. Push to your fork → 7. Open a Pull Request to the original repo → 8. Maintainers review & merge

GitHub Actions (Introduction)

Automated workflows that run on GitHub's servers whenever something happens in your repo — e.g. running tests on every push, or deploying automatically on merge to main. Defined in YAML files under `.github/workflows/`.

GitHub Projects (Introduction)

Kanban-style boards built into GitHub for tracking issues and PRs across columns like "To Do," "In Progress," and "Done."

GitHub Discussions

A forum-style space for open-ended conversation, Q&A, and announcements — separate from the more task-focused Issues.

PRO TIP

Your very first open-source contribution doesn't need to be code. Fixing a typo in a README via a Pull Request is a completely legitimate, low-pressure way to practice the entire fork → branch → PR workflow.

INTERVIEW QUESTION

Q: What's the difference between a fork and a clone?

A: A fork creates a copy of a repository under your own GitHub account. A clone downloads any repository (including your fork) onto your local machine.

Chapter Summary

- `.gitignore` and README.md are essential in every real project
- Fork → clone → branch → PR is the standard open-source contribution workflow
- Actions, Projects, and Discussions round out GitHub's collaboration toolkit

CHAPTER 18

The Patriot

The advanced developer mindset — habits that separate professionals from hobbyists

Professional Commit Habits

Junior developers write commits like `"fixed stuff"` or `"asdasd"`. Professionals follow a consistent convention that makes history genuinely useful months later.

Conventional Commit Messages

Prefix	Meaning	Example
<code>feat:</code>	New feature	<code>feat: add dark mode toggle</code>
<code>fix:</code>	Bug fix	<code>fix: correct off-by-one error in pagination</code>
<code>docs:</code>	Documentation only	<code>docs: update install instructions</code>
<code>refactor:</code>	Code change, no behavior change	<code>refactor: extract auth logic into helper</code>
<code>test:</code>	Adding or fixing tests	<code>test: add unit tests for cart total</code>

WHY THIS MATTERS

Good commit messages turn `git log` into a readable changelog. When a bug appears months later, a clear history lets you (or a teammate) understand *why* a change was made — not just what changed.

Branch Naming Conventions

```
feature/checkout-page
bugfix/login-crash
hotfix/payment-timeout
chore/update-dependencies
```

The prefix before the slash instantly tells anyone browsing the branch list what kind of work it contains.

Code Review Etiquette

- **As a reviewer:** critique the code, never the person; explain *why* a change is suggested, not just *what* to change
- **As an author:** don't take feedback personally; respond to every comment, even if just to say "done" or explain your reasoning
- Keep pull requests small — a 40-line PR gets reviewed carefully; a 2,000-line PR gets rubber-stamped

Open Source Contribution Mindset

- Read the project's `CONTRIBUTING.md` before submitting anything

- Start small — docs fixes, small bugs — before attempting large features
- Be patient; maintainers are often volunteers reviewing in their spare time

Industry Workflow Summary

Ticket assigned → branch created → small, focused commits → push → PR opened → automated tests run (CI) → peer review → approved → merged → deployed

PRO TIP

Interviewers frequently look at a candidate's GitHub commit history and PR discussions. Clean commit messages and thoughtful PR descriptions are a quiet but real signal of professionalism — long before you say a word in the interview.

INTERVIEW QUESTION

Q: Why do teams use conventional commit prefixes like `feat:` and `fix:` ?

A: They make history scannable at a glance, and many tools (like automated changelog and version-bump generators) parse these prefixes to automate releases.

Chapter Summary

- Use conventional prefixes (feat, fix, docs, refactor, test) in commit messages
- Name branches by type: feature/, bugfix/, hotfix/, chore/
- Small PRs, respectful reviews, and clean history are marks of a professional

CHAPTER 19

Git Cheat Sheet

Quick reference — every command from this book, grouped by category

Setup & Config

Command	Purpose
<code>git init</code>	Create a new repository
<code>git clone <url></code>	Download a repository
<code>git config --global user.name "Name"</code>	Set your name
<code>git config --global user.email "email"</code>	Set your email

Staging & Committing

Command	Purpose
<code>git status</code>	Show working tree state
<code>git add <file></code>	Stage a file
<code>git add .</code>	Stage everything
<code>git commit -m "msg"</code>	Commit staged changes
<code>git commit -am "msg"</code>	Stage tracked files + commit in one step

History & Inspection

Command	Purpose
<code>git log --oneline --graph</code>	Compact visual history
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes
<code>git show <hash></code>	Show details of one commit
<code>git blame <file></code>	Show who last changed each line

Branching

Command	Purpose
---------	---------

<code>git branch</code>	List branches
<code>git switch -c <name></code>	Create + switch to branch
<code>git switch <name></code>	Switch branches
<code>git merge <branch></code>	Merge a branch into current
<code>git branch -d <name></code>	Delete a merged branch

Remote & Sync

Command	Purpose
<code>git remote add origin <url></code>	Link a remote
<code>git push -u origin main</code>	Push + set upstream
<code>git push</code>	Push commits
<code>git pull</code>	Fetch + merge from remote
<code>git fetch</code>	Download remote changes without merging

Undo & Recovery

Command	Purpose
<code>git restore <file></code>	Discard unstaged changes
<code>git restore --staged <file></code>	Unstage a file
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes staged
<code>git reset --hard HEAD~1</code>	Undo last commit, discard changes
<code>git reflog</code>	View HEAD movement history for recovery
<code>git revert <hash></code>	Create a new commit that undoes a past commit

Stash, Tags & Rebase

Command	Purpose
<code>git stash</code>	Temporarily shelve changes
<code>git stash pop</code>	Restore + remove latest stash
<code>git tag -a v1.0.0 -m "msg"</code>	Create annotated tag
<code>git push origin --tags</code>	Push all tags

<code>git rebase main</code>	Replay commits on top of main
<code>git rebase --continue</code>	Continue after resolving a rebase conflict

PRO TIP

Print this chapter, or keep it open in a second monitor tab for your first few months of using Git professionally. Muscle memory builds fast once the four-stage model from Chapter 7 clicks.

CHAPTER 20

50 Interview Questions

Beginner, Intermediate, and Advanced — with answers

Beginner (1–18)

1. **What is Git?** A distributed version control system that tracks changes to files over time.
2. **What is a repository?** A project folder tracked by Git, containing all files and their full history.
3. **Difference between Git and GitHub?** Git is the tool; GitHub is a cloud platform that hosts Git repositories.
4. **What does `git init` do?** Initializes a new, empty Git repository in the current folder.
5. **What is the staging area?** A holding area where changes are placed before being committed.
6. **What does `git add .` do?** Stages all changed files in the current directory.
7. **What does `git commit` do?** Saves staged changes permanently to the local repository history.
8. **What is `git clone`?** Downloads a full copy of a remote repository, including history.
9. **What is a branch?** An independent line of development that diverges from the main codebase.
10. **What is the default branch usually called?** `main` (previously `master`).
11. **What does `git status` show?** The current state of staged, unstaged, and untracked files.
12. **What is a commit message?** A short description explaining what a commit changed and why.
13. **What is `.gitignore` for?** Listing files/folders Git should never track (e.g. secrets, build files).
14. **What does `git push` do?** Uploads local commits to a remote repository.
15. **What does `git pull` do?** Downloads and merges remote changes into your current branch.
16. **What is a remote?** A version of your repository hosted elsewhere, e.g. on GitHub, referenced as "origin."
17. **What's the difference between `git fetch` and `git pull`?** Fetch downloads changes without merging; pull downloads and merges automatically.
18. **What is a Pull Request?** A request to merge changes from one branch/fork into another, typically with review.

Intermediate (19–36)

19. **Difference between `git merge` and `git rebase`?** Merge preserves full history with a merge commit; rebase rewrites commits into a linear sequence.
20. **What causes a merge conflict?** Two branches changing the same lines of the same file in incompatible ways.
21. **How do you resolve a merge conflict?** Edit the file to remove conflict markers and choose the correct code, then add and commit.
22. **What is `git stash` used for?** Temporarily saving uncommitted changes so you can switch context cleanly.
23. **Difference between `git stash pop` and `apply`?** Pop removes the stash after restoring; apply keeps it in the list.
24. **What is a detached HEAD state?** When HEAD points directly to a commit instead of a branch, so new commits aren't attached to any branch.
25. **What does `git reset --soft` do?** Moves HEAD back but keeps changes staged.
26. **What does `git reset --hard` do?** Moves HEAD back and discards all changes, staged or not.
27. **What is `git revert`, and how does it differ from reset?** Revert creates a new commit undoing a past one, preserving history; reset rewrites history.

28. **What is `git reflog` ?** A local log of every HEAD movement, used to recover "lost" commits.
29. **What is a fast-forward merge?** A merge where the target branch simply moves forward, with no divergent history to combine.
30. **What's the difference between `git checkout` and `git switch` ?** Switch is the modern, focused command for changing branches; checkout is older and does multiple, overlapping jobs.
31. **What is a tag, and how does it differ from a branch?** A tag is a fixed pointer to one commit; a branch moves forward as you commit.
32. **What is `git cherry-pick` ?** Applies a specific commit from one branch onto another, without merging the whole branch.
33. **How do you rename a branch?** `git branch -m old-name new-name` .
34. **How do you undo the last commit but keep the changes?** `git reset --soft HEAD~1` .
35. **What is a shallow clone?** A clone that fetches only recent history (`git clone --depth 1`), saving time and space.
36. **What is `git blame` used for?** Showing who last modified each line of a file, and in which commit.

Advanced (37–50)

37. **What is the golden rule of rebasing?** Never rebase commits that have already been pushed and shared with others.
38. **How does Git store data internally?** As a content-addressable filesystem of objects (blobs, trees, commits) identified by SHA hashes.
39. **What is a blob in Git?** The object type Git uses to store the raw content of a file.
40. **What is a Git object's SHA used for?** A cryptographic fingerprint that uniquely identifies content and detects any corruption or change.
41. **What is `git bisect` ?** A binary-search tool that helps find the exact commit that introduced a bug.
42. **What's the difference between a merge commit and a squash merge?** A merge commit preserves every individual commit; a squash merge combines them into a single new commit.
43. **What happens during an interactive rebase (`git rebase -i`)?** You can reorder, edit, squash, or drop individual commits before they're replayed.
44. **What is a submodule?** A Git repository embedded inside another, pinned to a specific commit, used to include external dependencies.
45. **How do you completely remove a file from all of Git history?** Using tools like `git filter-repo` or the BFG Repo-Cleaner, followed by a force-push.
46. **What is a force-push, and why is it risky?** `git push --force` overwrites remote history; risky because it can silently discard teammates' commits.
47. **What is `--force-with-lease` ?** A safer force-push that fails if the remote has commits you haven't seen, preventing accidental overwrites.
48. **How does GitHub Actions relate to Git?** It triggers automated workflows (tests, builds, deploys) based on Git events like pushes and pull requests.
49. **What is trunk-based development?** A workflow where developers integrate small changes into main frequently, favoring short-lived branches over long-running ones.
50. **How would you recover work after accidentally deleting a branch?** Use `git reflog` to find the last commit hash on that branch, then recreate the branch from it with `git branch <name> <hash>` .

PRO TIP

Interviewers rarely expect a memorized textbook answer — they want to see that you understand *why* a command works the way it does. Practice explaining these out loud, not just reading them.

CHAPTER 21

Practical Projects

Apply everything you've learned with three hands-on projects

Project 1: Personal Portfolio

Goal: practice the core solo workflow — init, commit, branch, push.

1. Create a simple HTML/CSS portfolio site locally
2. `git init`, then commit your initial version
3. Create a repository on GitHub and push your project
4. Create a `feature/add-projects-section` branch, build it out, merge it back into main
5. Tag your first stable version: `git tag -a v1.0.0 -m "Initial portfolio release"`

Skills practiced: init, add, commit, branching, tagging, pushing.

Project 2: Team Collaboration Project

Goal: simulate a real multi-developer workflow. Recruit a friend, or use two GitHub accounts of your own.

1. One person creates a repository and adds the other as a Collaborator
2. Both clone the repository locally
3. Each person works on a separate feature branch simultaneously
4. Both open Pull Requests and review each other's code with comments
5. Deliberately edit the same line in both branches to trigger a merge conflict — then resolve it together

Skills practiced: branching, PRs, code review, conflict resolution, pull/push syncing.

Project 3: Open Source Contribution

Goal: make a real contribution to an existing public project.

1. Find a beginner-friendly repository (look for a "good first issue" label)
2. Fork the repository to your own GitHub account
3. Clone your fork locally, create a branch for your fix
4. Make your change, commit with a clear conventional message, push to your fork
5. Open a Pull Request to the original repository, following its contribution guidelines

Skills practiced: forking, the full open-source PR workflow, working with a codebase you don't own.

REAL-WORLD SCENARIO

These three projects mirror the actual arc of a developer's early career: solo projects first, then paid team collaboration, then giving back through open source. Completing all three gives you a genuinely credible GitHub profile to point to in interviews.

Chapter Summary

- Project 1 builds solo fundamentals: init, commit, branch, tag, push
- Project 2 builds collaboration skills: PRs, review, conflict resolution
- Project 3 builds real-world credibility through an open-source contribution

CHAPTER 22

Final Roadmap

From your first commit to professional developer

BEGINNER

- Install Git, configure identity
- init, add, commit, status, log
- Understand the four-stage workflow
- Push your first project to GitHub

↓

INTERMEDIATE

- Branching & merging
- Resolving merge conflicts
- Stash, tags, reflog
- Pull Requests and code review

↓

ADVANCED

- Rebase and interactive rebase
- Git internals (objects, SHA hashes)
- bisect, cherry-pick, filter-repo
- GitHub Actions and CI/CD

↓

OPEN SOURCE CONTRIBUTOR

- Fork → contribute → PR workflow on real projects
- Comfortable reading unfamiliar codebases
- Active, visible GitHub profile

↓

PROFESSIONAL DEVELOPER

- Trunk-based / team workflows at scale
- Mentors others in Git best practices
- Git is second nature — a tool, not an obstacle

Milestones to Track Your Progress

Milestone	You'll know you've hit it when...
First commit	You've pushed a real project to GitHub

First conflict resolved	You didn't panic, and fixed it calmly
First PR reviewed by someone else	You responded to feedback without taking it personally
First rebase	You cleaned up a branch's history before opening a PR
First open-source PR merged	Your code is now part of a project you don't own
First time explaining Git to someone else	You've genuinely internalized the four-stage model

CLOSING PRO TIP

Git mastery isn't about memorizing every command — it's about deeply understanding the four-stage workflow from Chapter 7. Once that mental model is second nature, every new command you encounter will make sense in context.

You've completed the Git & GitHub Handbook. Now go commit something.

ABOUT THE AUTHOR

Hammy Explains

Hammy Explains is a content creator focused on making technical skills genuinely understandable for beginners — through YouTube tutorials, hands-on projects, and guides like this one. Topics covered include:

- Python
- Git & GitHub
- SQL
- Machine Learning
- AI Engineering
- Agentic AI
- LangGraph
- FastAPI

Follow Along

YouTube: [Hammy Explains](#)

Instagram: [@hammy.explains](#)

Thank you for reading the Git & GitHub Handbook. Share it, star it, fork it — and most importantly, go build something.